

[Ask AI \(Beta\)](#)

The GDB developer's GNU Debugger tutorial, Part 1: Getting started with the debugger

April 30, 2021

[Keith Seitz](#)

Related topics: [C](#), [C#](#), [C++](#), [Linux](#), [Open source](#)

Related products: [Red Hat Enterprise Linux](#)

■ Table of contents:



This article is the first in a series demonstrating how to use the [GNU Debugger \(GDB\)](#) effectively to debug applications in [C](#) and [C++](#). If you have limited or no experience using GDB, this series will teach you how to debug your code more efficiently. If you are already a seasoned professional using GDB, perhaps you will discover something you haven't seen before.

In addition to providing developer tips and tricks for many GDB commands, future articles will also cover topics such as debugging optimized code, offline debugging (core files), and server-based sessions (*aka* `gdbserver`, used in container debugging).

Why another GDB tutorial?

The majority of GDB tutorials available on the web consist of little more than introductions to the basic `list`, `break`, `print`, and `run` commands. New GDB users just might as well read (or sing) the [official GDB Song](#)!

Instead of simply demonstrating a handful of useful commands, each article in this series will focus on one aspect of using GDB from the perspective of someone who develops GDB. I use GDB daily, and these tips and tricks are the ones that I (and many other advanced GDB users and developers) use to streamline our debugging sessions.

Because this is the first article in the series, allow me to follow the recommendation of the GDB Song and start at the very beginning: How to run GDB.

Compiler options

Let me get the (all-too-often-not-so) obvious out of the way: For the best debugging experience, build applications without optimization and with debugging information. That is trivial advice, but GDB's public freenode.net IRC channel (`#gdb`) sees these issues often enough that they warrant mentioning.

TL;DR: Don't debug applications with optimization if you can avoid it. Watch for a future article on optimization.

Optimization can cause GDB to behave in surprising ways if you are not aware of what might be happening "under the covers." I always use the C compiler option `-O0` (that's the letter *O* followed by the number zero) to build executables during the development cycle.

I also always have the toolchain emit debugging information. This is accomplished with the `-g` option. Specifying the exact debug format is no longer necessary (or desirable); DWARF has been the default debugging information format on GNU/Linux for many years. So ignore advice to use `-ggdb` or `-gdwarf-2`.

The one specific option worth adding is `-g3`, which tells the compiler to include debugging information about the macros (`#define F00 ...`) used in your application. These macros may then be used in GDB just like any other symbol in your program.

In short, for the best debugging experience, use `-g3 -O0` when compiling your code. Some environments (such as those using GNU autotools) set environment variables (`CFLAGS` and `CXXFLAGS`) that control the compiler's output. Check these flags to make sure that your invocations of the compiler enable the debugging environment you want.

For much more information about the impact of `-g` and `-O` on the debugging experience, see Alexander Oliva's treatise [GCC gOlogy: Studying the Impact of Optimizations on Debugging](#).

Startup scripts

Before we look at actually using GDB, something must be said about how GDB starts up and what script files it executes. Upon startup, GDB will execute the commands contained in a number of system and user script files. The location and order of execution of these files are as follows:

1. `/etc/gdbinit` (not on FSF GNU GDB): In many GNU/Linux distributions, including Fedora and [Red Hat Enterprise Linux](#), GDB looks first for the system default initialization file and executes commands contained therein. On Red Hat-based systems, this file executes any script files (including [Python](#) scripts) installed in `/etc/gdbinit.d`.
2. `$HOME/.gdbinit`: GDB will then read the user's global initialization script from the home directory, if this file exists.
3. `./gdbinit`: Finally, GDB will look for a startup script in the current directory. Think of this as an application-specific customization file where you can add per-project user-defined commands, pretty-printers, and other customizations.

All of these startup files contain GDB commands to execute, but they may also include Python scripts as long as they are prefaced with the `python` command, e.g., `python print('Hello from python!')`.

My `.gdbinit` is actually quite simple. Its most essential lines enable command history so that GDB remembers a given number of commands

that were executed from a previous session. This is analogous to the shell's history mechanism and `.bash_history`. The entire file is:

```
set pagination off
set history save on
set history expansion on
```

The first line turns off GDB's built-in paging. The next line enables saving the history (to `~/.gdb_history` by default), and the final line enables shell-style history expansion with the exclamation point (!) character. This option is normally disabled because the exclamation point is also a logical operator in C.

To prevent GDB from reading initialization files, give it the `--nx` command-line option.

Getting help in GDB

There are several ways to get help using GDB, including extensive—if dry—[documentation](#) explaining every little switch, knob, and feature.

GDB community resources

The community offers help to users in two places:

- Via email: The [GDB mailing list](#)
- Via IRC: #gdb on [libera.chat](#)

However, because this article is about *using* GDB, the easiest way for users to get help with a command is to use GDB's built-in help system, discussed next.

Accessing the help system

Access GDB's built-in help system via the `help` and `apropos` commands. Don't know how to use the `printf` command? Ask GDB:

```
(gdb) help printf
Formatted printing, like the C "printf" function.
Usage: printf "format string", ARG1, ARG2, ARG3,
..., ARGN
This supports most C printf format
specifications, like %s, %d, etc.
(gdb)
```

`help` accepts the name of any GDB command or option and outputs usage information for that command or option.

Like all GDB commands, the `help` command supports tab completion. This is perhaps the most useful way to figure out what types of arguments many commands accept. For instance, entering `help show ar` and pressing the tab key will prompt you for a completion:

```
(gdb) help show ar
architecture  args          arm
(gdb) help show ar
```

GDB leaves you at the command prompt ready to accept further refinement of the input. Adding `g` to the command, followed by a tab, will complete to `help show args`:

```
(gdb) help show args
Show argument list to give program being debugged
when it is started.
Follow this command with any number of args, to
be passed to the program.
(gdb)
```

Don't know the exact name of the command you're looking for? Use the `apropos` command to search the help system for specific terms. Think of it as grepping the built-in help.

Now that you know how and where to find help, we're ready to move on to starting GDB (finally).

Starting GDB

Unsurprisingly, GDB accepts a large number of command-line options to change its behavior, but the most basic way to start GDB is to pass the application's name to GDB on the command line:

```
$ gdb myprogram
GNU gdb (GDB) Red Hat Enterprise Linux 9.2-2.el8
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later
<http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and
redistribute it.
There is NO WARRANTY, to the extent permitted by
law.
Type "show copying" and "show warranty" for
details.
This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration
details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation
resources online at:

<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands
related to "word"...
```

```
Reading symbols from /home/blog/myprogram...  
(gdb)
```

GDB starts up, prints out some version information (GCC Toolset 10 shown), loads the program and its debug information, and displays copyright and help messages, ending with the command prompt, `(gdb)`. GDB is now ready to accept input.

Avoiding messages: The `-q` or `--quiet` option

I've seen GDB's startup message thousands of times, so I suppress (or "quiet") it with the `-q` option:

```
$ gdb -q myprogram  
Reading symbols from /home/blog/myprogram...  
(gdb)
```

That's much less to read. If you are really new to GDB, you might find the full startup messaging useful or soothing, but after a while, you'll also alias `gdb` in your shell to `gdb -q`. If you do need the suppressed information, use the `-v` command-line option or the `show version` command.

Passing arguments: The `--args` option

Programs often require command-line arguments. GDB offers multiple ways to pass these to your program (or "inferior," in GDB parlance). The two most useful ways are to pass application arguments via the `run` command or at startup via the `--args` command-line option. If your application is normally started with `myprogram 1 2 3 4`, simply preface this with `gdb -q --args` and GDB will remember how your application should be run:

```
$ gdb -q --args myprogram 1 2 3 4  
Reading symbols from /home/blog/myprogram...  
(gdb) show args  
Argument list to give program being debugged when
```

```
it is started is "1 2 3 4".
(gdb) run
Starting program: /home/blog/myprogram 1 2 3 4
[Inferior 1 (process 1596525) exited normally]
$
```

Attaching to a running process: The --pid option

If an application is already running and gets "stuck," you might want to look inside to find out why. Just give GDB the process ID of your application with

```
--pid :
```

```
$ sleep 100000 &
[1] 1591979
$ gdb -q --pid 1591979
Attaching to process 1591979
Reading symbols from /usr/bin/sleep...
Reading symbols from .gnu_debugdata for
/usr/bin/sleep...
(No debugging symbols found in .gnu_debugdata for
/usr/bin/sleep)
Reading symbols from /lib64/libc.so.6...
Reading symbols from
/usr/lib/debug/usr/lib64/libc-2.31.so.debug...
Reading symbols from /lib64/ld-linux-x86-
64.so.2...
Reading symbols from /usr/lib/debug/usr/lib64/ld-
2.31.so.debug...
0x00007fc421d5ef98 in __GI__clock_nanosleep
(requested_time=requested_time@entry=0,
remaining=remaining@entry=0x0)
    at
../sysdeps/unix/sysv/linux/clock_nanosleep.c:28
```



```
28         return SYSCALL_CANCEL (nanosleep,  
requested_time, remaining)  
(gdb)
```

With this option, GDB automatically loads symbols for programs that have build ID information, such as distribution-supplied packages, and interrupts the program so that you can interact with it. Look for more on how and where GDB finds debug information in a future article.

Following up on a failure: The `--core` option

If your process aborted and dumped core, use the `--core` option to tell GDB to load the core file. If the core file contains the build ID of the aborted process, GDB automatically loads that binary and its debugging information if it can. Most developers, however, need to pass an executable to GDB with this option:

```
$ ./abort-me  
Aborted (core dumped)  
$ gdb -q abort-me --core core.2127239  
Reading symbols from abort-me...  
[New LWP 2127239]  
Core was generated by `./abort-me'.  
Program terminated with signal SIGABRT, Aborted.  
#0  __GI_raise (sig=sig@entry=6) at  
../sysdeps/unix/sysv/linux/raise.c:50  
50         return ret;  
(gdb)
```

Tip: Can't find a core file? On GNU/Linux systems using systemd, check `ulimit -c` to see whether the shell is preventing programs from creating core files. If the value is `unlimited`, use `coredumpctl` to find the core file. Alternatively, run `sysctl -w kernel.core_pattern=core` to configure systemd to output core files named `core.` `PID`, as I have for the previous example.

Expedited command execution: The `--ex`, `--iex`, `--x`, and `--batch` options

I often run GDB commands repeatedly from the shell to test for problems or run scripts. These command-line options help facilitate that. Most users will use (multiple) `--ex` arguments to specify commands to run at startup to recreate a debugging session, e.g., `gdb -ex "break some_function if arg1 == nullptr" -ex r myprogram`.

- `--ex CMD` runs the GDB command `CMD` after the program (and debug information) is loaded. `--iex` does the same, but executes `CMD` before the specified program is loaded.
- `-x FILE` executes GDB commands from `FILE` after the program is loaded and `--ex` commands execute. I use this option most often if I need a lot of `--ex` arguments to reproduce a specific debugging session.
- `--batch` causes GDB to exit immediately at the first command prompt; i.e., after all commands or scripts have run. Note that `--batch` will silence even more output than `-q` to facilitate using GDB in scripts:

```
$ # All commands complete without error
$ gdb -batch -x hello.gdb myprogram
Reading symbols from myprogram...
hello
$ echo $?
0
$ # Command raises an exception
$ gdb -batch -ex "set foo bar"
No symbol "foo" in current context.
$ echo $?
1
$ # Demonstrate the order of script execution
```

```
$ gdb -x hello.gdb -iex 'echo before\n' -ex 'echo
after\n' simple
GNU gdb (GDB) Red Hat Enterprise Linux 9.2-2.el8
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later
<http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and
redistribute it.
There is NO WARRANTY, to the extent permitted by
law.
Type "show copying" and "show warranty" for
details.
This GDB was configured as "x86_64-redhat-linux-
gnu".
Type "show configuration" for configuration
details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation
resources online at:

<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands
related to "word"...
before
Reading symbols from simple...
hello
after
(gdb)
```

Next up

In this article, I've shared details about how GDB starts up, reads scripts (and when it reads scripts), and several startup options commonly used by advanced GDB users.

The next article in the series will take a small detour to explain what debugging information is, how to inspect it, where GDB looks for it, and how to install it in distribution-supplied packages.

Do you have a suggestion or tip related to GDB scripts or startup, or a suggestion for a future topic about how to use GDB? Leave a comment on this article and share your idea with us.

Last updated: February 27, 2024

Related Posts

[Printf-style debugging using GDB, Part 1](#)

[Printf-style debugging using GDB, Part 2](#)

[Printf-style debugging using GDB, Part 3](#)

Recent Posts

[Protect data offloaded to GPU-accelerated environments with OpenShift sandboxed containers](#)

[Case study: Measuring energy efficiency on the x64 platform](#)

[How to prevent AI inference stack silent failures](#)

Preventing GPU waste: A guide to JIT checkpointing with Kubeflow Trainer on OpenShift AI

How to manage TLS certificates used by OpenShift GitOps operator



Platforms

Build

Quicklinks

Communicate

RED HAT DEVELOPER

Build here. Go anywhere.

We serve the builders. The problem solvers who create careers with code.

Join us if you're a developer, software engineer, web designer, front-end designer, UX designer, computer scientist, architect, tester, product manager, project manager or team lead.

Sign me up →



[About Red Hat](#)

[Jobs](#)

[Events](#)

[Locations](#)

[Contact Red Hat](#)

[Red Hat Blog](#)

[Inclusion at Red Hat](#)

[Cool Stuff Store](#)

[Red Hat Summit](#)

[© 2026 Red Hat](#)

[Privacy statement](#)

[Terms of use](#)

[All policies and guidelines](#)

[Digital accessibility](#)